



General Operations Over Basic Embedded Radio

A flexible digital communications protocol.

Abdul Zia, Arizona State University

November 26th, 2025

Version 1.00.0

1 Introduction

The General Operations Over Basic Embedded Radio (abbreviated GOOBER) protocol is designed for use in vehicle avionics. The GOOBER protocol dictates how command and control exchanges should take place between devices communicating on the same frequency, channel, or bus.

GOOBER is designed to be used through various physical layers such as LoRa, 4G LTE, TCP/UDP, Bluetooth, etc. to create a network of interconnected devices. While networking multiple devices is the end goal, GOOBER's primary purpose is to facilitate communications between devices on a shared communication path.

Unless otherwise specified, GOOBER is little-endian everywhere.

2 Packet Structure

At its core, GOOBER is just a structure of bytes arranged in a meaningful manner. The structure of a GOOBER packet with N payload bytes is shown below.

Name	DEV_ID	DEV_MODE	SEQ_ID	MSG_CLS	Payload
Size (bytes)	1	1	1	1	N
Offset (bytes)	0	1	2	3	4

2.1 DEV_ID Byte

2.1.1 Unique device identifiers

Each device on a GOOBER network has a unique device identifier (DEV_ID) represented as an unsigned 8 bit integer (0-255). Users implementing the GOOBER protocol are responsible for preventing duplicate device identifiers in their network.

2.1.2 Segmentation of device identifier range

It's recommended to segment device IDs based on the different physical layers present in a network. For example, devices numbered 0-16 can be allocated for LoRa radios, devices numbered 17-32 can be allocated for Bluetooth devices, and so on.

2.1.3 DEV_ID byte arrangement

Bit	0	1	2	3	4	5	6	7
Value	ID[0]	ID[1]	ID[2]	ID[3]	ID[4]	ID[5]	ID[6]	ID[7]

2.2 DEV_MODE Byte

GOOBER packets include device mode (DEV_MODE) bits that convey important information about the packet's features.

2.2.1 Transmission Mode

Transmissions on the GOOBER network can be simplex, half-duplex, or full duplex. Transmission mode 1 indicates a simplex link. Transmission mode 2 indicates a half-duplex link. Transmission mode 3 indicates a full-duplex link. Transmission mode 0 is an invalid/error mode. Devices in simplex will not listen for packets and shall continually transmit while their device mode is 1.

2.2.2 Intent Bit

In half-duplex transmissions, the transmitter may want to transmit another packet before switching roles in the communication exchange. The intent bit indicates this behavior. Receivers in half-duplex communication shall continue to listen for the next packet if they receive a packet with an intent bit of 1. An intent bit of 0 in half-duplex communication indicates that the transmitter will listen for packets immediately after transmitting.

2.2.3 Size Bit

Certain physical layers may already transmit the size of the packet, so including the size of the packet may be redundant. If the size bit is 1, a 16-bit unsigned integer representing the length of the payload data in bytes will be appended to the front of the payload data.

2.2.4 Checksum Bit

Certain physical layers like LoRa have CRC/Checksums built-in, so including a checksum in the packet may be redundant. If the checksum bit is 1, a 16 bit fletcher checksum is appended to the end of the payload data. The checksum is calculated over the entire length of the serialized packet before transmission. This checksum is included in the calculation of the total size of the payload, if that is included (see Size Bit above).

2.2.5 Command-only Bit

Certain command-and-control exchanges may not require payload data. If the command-only bit is 1, there is no payload data included in the packet.

If the checksum bit is 1, the checksum bytes are the only payload data transmitted. The size bit shall be 0 if the command-only bit is 1, since the packet size can only be 4 (no checksum bytes) or 6 bytes (checksum bytes) in length.

2.2.6 DEV_MODE byte arrangement

Bit	0	1	2	3	4	5	6	7
Value	T.Mode LSB	T.Mode MSB	Intent Bit	Size Bit	Checksum Bit	Rsvd.	Rsvd.	Rsvd.

- Bit 0: Transmission mode LSB.
- Bit 1: Transmission mode MSB.
- Bit 2: Intent bit.
- Bit 3: Size bit.
- Bit 4: Checksum bit.
- Bit 5: Command-only bit.
- Bits 6-7: Reserved for future use.

2.3 SEQ_ID Byte

The SEQ_ID is an unsigned 8-bit integer that increments with each transmission. This allows receivers to detect if any packets were dropped or received out of order during communication.

The sequence ID wraps around from 255 to 0 after reaching its maximum value. Each device maintains its own sequence counter that increments with every packet transmitted. The SEQ_ID should not be used to determine the timestamp of a transmission. It is not time dependent.

2.3.1 SEQ_ID byte arrangement

Bit	0	1	2	3	4	5	6	7
Value	SEQ[0]	SEQ[1]	SEQ[2]	SEQ[3]	SEQ[4]	SEQ[5]	SEQ[6]	SEQ[7]

2.4 MSG_CLS Byte

The message class (MSG_CLS) byte is an unsigned 8-bit integer ranging from 0x01 to 0xFE that indicates the type of the payload/command being sent. Message classes are stored in a lookup table on each device in the network and are application-dependent.

Special values:

- 0x00: Reserved.
- 0xFF: Reserved.

2.4.1 MSG_CLS byte arrangement

Bit	0	1	2	3	4	5	6	7
Value	CLS[0]	CLS[1]	CLS[2]	CLS[3]	CLS[4]	CLS[5]	CLS[6]	CLS[7]

2.5 Payload Bytes

Each message class (MSG_CLS) has an accompanying payload type N bytes in length. This payload type is dependent on the user's application.

2.5.1 Size Bytes

If the packet's size bit (DEV_MODE[3]) is 1, a 16-bit unsigned integer N representing the length of the payload data in bytes will be appended to the front of the payload data. If the size bit is 1, the length of the payload data in bytes cannot exceed 65535.

2.5.2 Checksum Bytes

If the checksum bit (DEV_MODE[4]) is 1, a 16 bit fletcher checksum is appended to the end of the payload data. The checksum is calculated over the entire length of the serialized packet before transmission. This checksum is included in the calculation of the total size of the payload, if that is included (see Size Bytes above).

2.5.3 Command-Only Exception

If the command-only bit (DEV_MODE[5]) is 1, there is no payload data included in the packet ($N=0$).

If the checksum bit is 1, the checksum bytes are the only payload data transmitted. The size bit shall be 0 if the command-only bit is 1, since the packet size can only be 4 (no checksum bytes) or 6 bytes (checksum bytes) in length.

2.5.4 Payload Byte Arrangement, Size and Checksum Included

Byte Offset	Field	Size (bytes)
0-1	Size	2
$2-(1+N)$	Payload Data	N
$(2+N)-(3+N)$	Checksum	2

2.5.5 Payload Byte Arrangement, Size Only

Byte Offset	Field	Size (bytes)
0-1	Size	2
2-(1+N)	Payload Data	N

2.5.6 Payload Byte Arrangement, Checksum Only

Byte Offset	Field	Size (bytes)
0 - (N-1)	Payload Data	N
N-(1+N)	Checksum	2

2.5.7 Payload Byte Arrangement, No Size or Checksum

Byte Offset	Field	Size (bytes)
0 - (N-1)	Payload Data	N

3 Appendix

3.1 Fletcher Checksum

Example code for calculating the 16 bit fletcher checksum is below:

```
uint16_t Fletcher16( uint8_t *data, int count )
{
    uint16_t sum1 = 0;
    uint16_t sum2 = 0;
    int index;

    for ( index = 0; index < count; ++index )
    {
        sum1 = (sum1 + data[index]) % 255;
        sum2 = (sum2 + sum1) % 255;
    }

    return (sum2 << 8) | sum1;
}
```